

Quiz 2

MSDM 5001 - Introduction to Computational and Modeling Tools (Fall 2025)

Date: 30 November 2025

Time allowed: 1 hour 15 minutes

Instructions:

1. Make sure you have stable internet connection.
2. Make sure you have all relevant softwares (`numpy`, `mpi4py`, Cython with compatible C/C++ compilers) properly installed in one environment.
3. There will be both written and coding components. All written answers (including answers to MCQs) must be submitted as one single PDF file on Canvas.
4. For written answers, you must show steps to receive full credits.
5. For coding questions, you must follow format of the skeleton codes provided.
6. If you submit scanned copy of handwritten answers, make sure they are legible. Otherwise, points may be deducted.
7. You are not allowed to share anything regarding this quiz until after 12 p.m. on 1 December. Failure to comply may lead to penalty.
8. You are not allowed to use any generative AI tools in completing this quiz. **Offenders will receive zero points for this quiz, and may face disciplinary actions.**
9. You may use the internet for looking up documentation only.
10. If you encounter any issues, or would like to have clarifications on the questions, please contact phycsan@ust.hk **immediately**. In particular, if there are issues preventing you from submitting on Canvas, you must send all files to me before the deadline. Failure to do so may lead to deduction in points.
11. By submitting, you confirm that all work is your own and complies with the university's academic integrity policy.

Problem 1: Multiple Choice Questions (25 points)

Select the best answer for each question. Each question is worth 5 points.

1. Which of the following MPI codes should be the most efficient for sending a large numeric array?

- (A) `comm.send(list(range(10**7)))`
- (B) `comm.send(np.arange(10**7))`
- (C) `comm.Send(np.arange(10**7))`
- (D) `comm.send(bytearray(10**7))`

ANSWER: _____

2. What is the bisection bandwidth of a fully connected graph with n vertices?

- (A) n^2
- (B) $n^2/4$
- (C) $2n^2$
- (D) $\sqrt{n}$

ANSWER: _____

3. Consider a two-dimensional **numpy** array `a`, which of the following operations must allocate a temporary buffer (i.e., it cannot perform the operation purely as a simple in-place update of the original memory)?

- (A) `a[m] += 1` where `m` is boolean mask
- (B) `a[:, :2] += 1`
- (C) `a[:, :-1] += 1`
- (D) `a.T += 1`

ANSWER: _____

4. You inherited a Cython codebase that

- Adds `cdef` types everywhere;
- Calls multiple Python functions inside inner loops; and
- Uses `cpdef` for the public API to be called from an external Python file

You find the code's performance disappointing. Where is the most impactful optimization likely to come from?

- (A) Switching all `cpdef` to `cdef`.
- (B) Eliminating Python calls in the loops by using C-level calls
- (C) Setting `boundscheck=False` globally.
- (D) Replacing all the Python lists in the code with `numpy` arrays.

ANSWER: _____

5. Consider a qubit in the state $N(e^{i\theta/2} |0\rangle - e^{-i\theta/2} |1\rangle)$, where N is some normalization constant. Suppose we measure an observable A given by the following matrix in the computational basis:

$$A = \begin{pmatrix} 1 & i \\ -i & 1 \end{pmatrix}.$$

For which value of θ is the expected value of A minimized?

- (A) $\pi/2$
- (B) $\pi/4$
- (C) $-\pi/4$
- (D) $-\pi/2$

ANSWER: _____

Problem 2: Simulating Percolation on a 2D Grid [30 points]

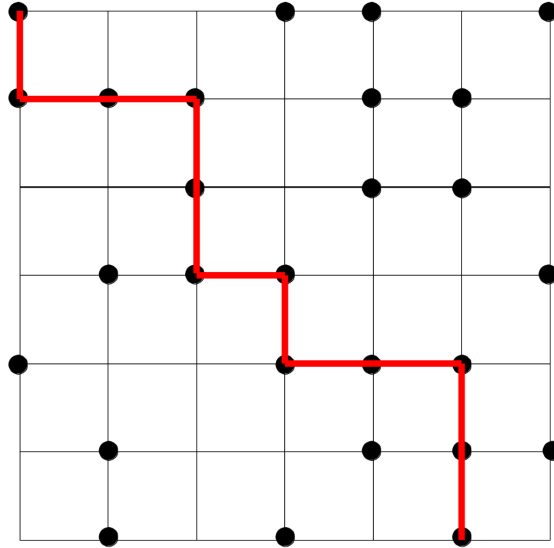


Figure 1: A 7-by-7 two-dimensional grid with a percolating path in red. Black solid dots are open, whereas unlabelled lattice points are closed.

Consider an $n \times n$ square grid, where n refers to the number of lattice points per row/-column. On this grid:

- Each lattice point is *independently* “open” with probability p , or “closed” with probability $1 - p$
- Two lattice points are considered *connected* if they are both open, and are vertically or horizontally adjacent
- We say the grid “percolates” if there is a connected path between the top and bottom edge

The goal of this problem is to simulate the system above, and estimate for given values of p and n what the probability of percolation is. Skeleton codes are provided.

Let us represent the lattice points as an $n \times n$ **numpy** binary array, where 1 means the lattice point is open, and 0 means the lattice point is closed (see fig. 1)

- (a) Using `numpy.random.choice`, write a function `gen_grid(p, n)` that returns an $n \times n$ numpy array where each element is either 0 with probability $1 - p$ or 1 with probability p . Explicit for loops are forbidden. You may refer to the official documentation [here](#).

Submission instructions: Complete the function `gen_grid` in “percolate.py”

- (b) Given a grid generated by `gen_grid(p, n)` you wrote in (a), write a function `check_percolate(grid)` in Cython that returns `True` if the grid percolates, and `False` otherwise.

One way to implement this is to use something called “depth-first-search”. If you are not familiar with it, here’s another idea:

- Start from the first row in the grid. Mark all 1s as “reachable”.
- For every point in the grid, mark it as “reachable” if its value is 1, and is connected (horizontally or vertically) to any already-reachable point.
- Repeat until no new points become reachable
- The grid percolates if and only if there is any reachable point in the last row.

You are only allowed to use one of these two algorithms above, and it must be implemented in Cython.

Submission instructions: Fill in the missing codes in “check_percolate.pyx”. We have also provided a corresponding “setup.py” for you to compile your code, but you do not need to submit it.

- (c) To estimate the probability of percolation for given values of p and n , we run the functions `gen_grid(p, n)` and `check_percolate(grid)` $N = 10000$ times, and count the fraction of runs for which we have percolating grids.

Write an MPI program using `mpi4py` to parallelize this task. Each MPI process should handle a subset of the 10,000 simulations, and count how many simulations yield a percolating grid. Finally, use `reduce` or `gather` to combine results and get the final, total count. **Make sure that your solutions will work for all possible values of N and number of MPI processes.**

Notice that you need to have your Cython code in (b) compiled in order to use it here. If you were unable to finish parts (a) and (b), you may also call those functions as if they have been implemented.

To check your answer, when $n = 50$ and $p = 0.6$, the percolation probability should be about 0.6.

Submission instructions: Complete the function `percolation_probability` in “percolate.py”

Problem 3: Quantum Correlation (10 points)

Consider a pair of qubits in the state

$$|\Psi\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle), \quad (1)$$

where Alice holds the first qubit, and Bob holds the second.

Suppose Alice can choose to measure either

$$A_1 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \text{ or } A_2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad (2)$$

and Bob can measure either

$$B_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \text{ or } B_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ -1 & -1 \end{pmatrix}. \quad (3)$$

Let's define the *correlation* between Alice's and Bob's measurements, $E(A_i, B_j)$, as the expected value of $a_i \cdot b_j$, where a_i and b_j denote the outcomes of measuring observables A_i and B_j on Alice's and Bob's qubits respectively. Compute

$$S = E(A_1, B_1) + E(A_2, B_1) - E(A_1, B_2) + E(A_2, B_2). \quad (4)$$

You must show all steps to receive full credits.

Problem 4: Two Circuit Identities (10 points)

Prove the following two identities:

